

Sentiment Analysis of Online Shop Reviews: A Comparative Study Using Logistic Regression, Naïve Bayes, XGBoost, and BERT

1st Jason Montolalu
Computer Science Department
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia
jason.montolalu@binus.ac.id

2nd Rakha Adinata Xavier
Computer Science Department
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia
rakha.zavier@binus.ac.id

3rd Muhammad Ridho Prakoso
Computer Science Department
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia
muhammad.prakoso010@binus.ac.id

4th Catherine Zaneta Adji
Computer Science Department
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia
catherine.adji@binus.ac.id

5th Mohammad Faisal Riftiarrasyid
Computer Science Department
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia
mohammad.riftiarrasyid@binus.ac.id

Abstract— The exponential growth of online shopping has led to an overwhelming volume of customer reviews, making manual sentiment assessment impractical for sellers seeking to understand customer satisfaction. This study presents a comparative analysis of four text classification approaches for sentiment analysis of online shop reviews: Logistic Regression, Multinomial Naive Bayes, XGBoost, and BERT. Using the BanglaEcomReviewCorpus dataset, consisting of 8,587 English-translated reviews labeled as positive, negative, or neutral, we implemented a TF-IDF feature extraction pipeline for the three classical machine learning models, while BERT was fine-tuned directly on minimally processed text to leverage its contextual subword representations. Hyperparameters for each classical model were optimized through grid and randomized search with five-fold cross-validation, and all models were evaluated on a held-out test set using accuracy, precision, recall, and macro-averaged F1-score. Experimental results show that BERT achieved the highest performance with a macro F1-score of 94.4%, outperforming the best classical model, Logistic Regression, by approximately 12 percentage points (82.0%). Among the classical models, Logistic Regression and Naive Bayes performed comparably, both surpassing XGBoost (79%), whose performance was likely constrained by the high-dimensional sparsity of TF-IDF features. We integrated all four trained models into CheckShop, a publicly accessible Streamlit web application that allows sellers to obtain real-time, multi-model sentiment predictions for English-language product reviews. Our experimental, and website code can be accessed at this link: https://github.com/RakhaZvy/CheckShop_APPLICATION.git

Keywords—sentiment analysis, online shop reviews, Logistic Regression, Naive Bayes, XGBoost, BERT, TF-IDF, comparative study

I. INTRODUCTION

The rapid growth of e-commerce platforms has changed the way people purchase products, ranging from necessities to wish lists. Along with this growth, customers increasingly share their experiences by leaving comments and reviews regarding the experience or the products they have purchased, providing valuable feedback for both potential buyers and

sellers. These reviews often contain opinions, emotions, or evaluations that can influence purchasing decisions and help sellers understand customer satisfaction, allowing them to reflect and improve both their products and services [1].

However, the large volume of online comments makes it difficult to manually analyze customer opinions efficiently. Resulting in businesses struggling to understand public views of either their products or services [2]. This matter creates a need for automated methods that can process textual data effectively.

As a result, researchers have been exploring sentiment analysis as a solution to process reviews, gaining valuable product or service insights from customers. Sentiment analysis is one of the Natural Language Processing (NLP) fields that focuses on identifying and classifying opinions into classes, such as positive, negative, and neutral sentiments [3]. By identifying customer sentiment, consumer behavior can be better understood, and decisions that are based on public opinion can be easily made.

Recent advances in machine learning have further improved the performance of sentiment analysis systems. Machine learning algorithms like Naive Bayes and Support Vector Machine (SVM) have been often found in sentiment classification tasks, achieving promising results [4]. More recently, deep learning models such as Bidirectional Encoder Representations from Transformers (BERT) with its transformer-based architecture, have gained attention due to their ability to understand contextual relationships between each word in a sentence. [5] BERT achieves superior performance compared to traditional machine learning models and improving performance in various sentiment analysis tasks [6].

This project focuses on sentiment analysis of online shop comments to automatically determine the sentiment behind each comment. The results can help sellers monitor customer satisfaction, identify each product's strengths and weaknesses [7], and support data-driven business decisions.

II. RELATED WORKS

A. Sentiment Analysis in E-Commerce

Sentiment analysis has been widely applied in e-commerce platforms to understand customer's views on their experience and for sellers to improve business strategies. [8] conducted sentiment analysis on Shopee, a Singaporean e-commerce, using the Naive Bayes Classifier. This study applied a few preprocessing techniques like tokenization, filtering, and stemming, also followed by TF-IDF for feature extraction. Their model achieved an accuracy of 76.92%, Recall 80%, and Precision 74.07%.

A study conducted in 2025 developed a sentiment classification model for e-commerce reviews using Support Vector Machine (SVM) and TF-IDF for feature extraction. This study removed stop words that do not contribute much to sentiment classification. TF-IDF transforms review texts into numerical vectors, then the SVM classifier trained on labeled review data to categorize the reviews into 3 classes, such as positive, negative, and neutral. This model achieved an accuracy of 80.31% [9].

B. Feature Extraction and Traditional Machine Learning

Traditional machine learning algorithms such as Naive Bayes, Support Vector Machine, and Logistic Regression remain widely used in sentiment analysis due to their strong performance when combined with effective feature extraction techniques, with TF-IDF among the most common.

Naive Bayes offers strong computational efficiency and can perform well even with limited training data [8]. However, this algorithm assumes that features are independent, which often comes unrealistic in natural language where words in a sentence should be highly correlated. This limitation reduces its effectiveness in analyzing complex sentiment patterns.

Support Vector Machine (SVM) has also often been found in sentiment analysis. Combining TF-IDF and SVM showed effective classification of reviews into positive, negative, and neutral sentiments [9]. SVM is generally more capable of handling high-dimensional text data, but its performance can be sensitive to parameter selection and may require greater computational resources on large datasets [10].

Another approach that is widely used is Logistic Regression, which demonstrated strong performance in sentiment classification [11]. Logistic Regression offers a good balance of interpretability and predictive performance, though it remains constrained by its linear decision boundary [12].

Researchers have also explored eXtreme Gradient Boosting (XGBoost), an ensemble algorithm based on multiple decision trees [13], for sentiment analysis tasks. However, evidence on its advantage over linear models is mixed: a comparative study on multilingual Twitter sentiment classification using XGBoost and Logistic Regression on a balanced dataset found that Logistic Regression was, in fact, the best-performing classifier, with an accuracy of 81%, despite the same study noting that “the XGBoost classifier gives better results for the Twitter dataset than the logistic regression model” elsewhere in its discussion an internal inconsistency in the source itself [14]. This suggests that XGBoost's added model complexity does not always translate into a clear, reproducible accuracy advantage over simpler linear classifiers on short, sparse text data.

C. Linear Classifier and Ensemble Learning

Another widely used approach is Logistic Regression, which has demonstrated strong performance in sentiment classification tasks [11]. A direct comparison between Naive Bayes and Logistic Regression on Indonesian marketplace reviews (Shopee and Lazada) found that Logistic Regression achieved the best result on the Shopee dataset, with an accuracy of 84.58%, precision of 84.66%, and recall of 84.63% [15]. Logistic Regression offers a good balance of interpretability and predictive performance, though it remains constrained by its linear decision boundary [12].

D. Comparative Frameworks and System Deployment

While the studies above each contribute valuable benchmarks, most evaluate only two algorithms at a time within a single domain — Naive Bayes alone on Shopee game-product reviews [15], Naive Bayes against Logistic Regression on rating-labeled marketplace data [16], or XGBoost against linear and ensemble baselines on Western e-commerce clothing reviews [14] — rather than directly comparing Naive Bayes, Logistic Regression and XGBoost together within a single, consistent TF-IDF pipeline applied to local online shop comment data. Furthermore, while these studies establish strong experimental benchmarks, none translate their findings into a deployed, end-to-end system accessible to sellers in real time. This project addresses both gaps: it conducts a systematic, side-by-side comparison of Naive Bayes, Logistic Regression, and XGBoost using TF-IDF vectorization on online shop comment data and integrates the best-performing model into a fully functional, publicly accessible web application — bridging the gap between experimental benchmarking and practical, real-world utility for e-commerce sellers.

III. METHODOLOGY

A. Experimental Workflow

This research was conducted through several main stages that form the foundation of our experimental pipeline. The research flow begins with data collection and concludes with the model's implementation into an application or its deployment as a website. Our experimental pipeline is illustrated in Fig. 1 below.

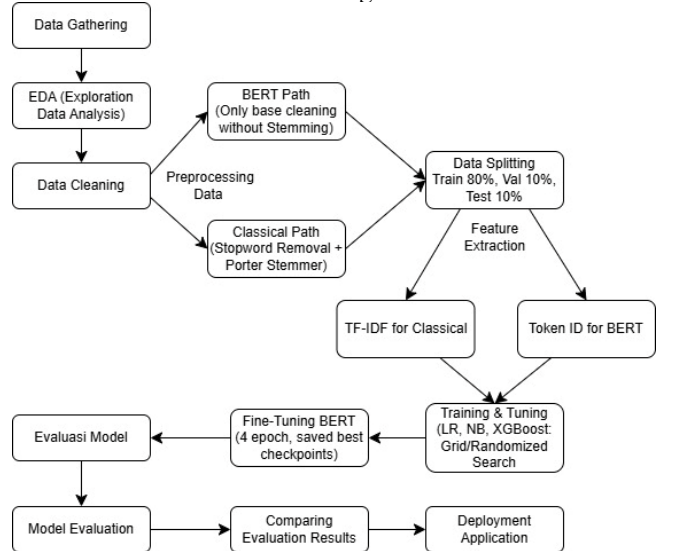


Fig. 1. Research Methodology and Experimental Pipeline

B. Data Collection

The dataset is sourced from [17], consisting of 8685 reviews in Bangla, their sentiment, and equivalent English translation. The dataset consists of 34.7% positive, 33.2% negative, and 32.1% neutral reviews.

C. EDA and Data Preprocessing

The dataset was first processed to drop the Bangla reviews since only the English translation of said reviews will be used for training, validation, and testing. The dataset is balanced across the three sentiments, positive being 34.7%, negative being 33.2%, and neutral being 32.1% before any preprocessing. Since the dataset is near balanced, no class-balancing was done during training, validation, or testing. Rows with null values in the English review or sentiment were dropped. The dataset was found to contain 88 fully duplicated rows and among these duplicated rows, 5 contained the same English review yet conflicting sentiment. Both the fully duplicated rows and label conflicting entries were removed before modeling. No empty values were found, but non-ASCII characters were present in 6 rows. The non-ASCII character was a recurring zero-width space character (U+200B), it was addressed during cleaning. Reviews are short, single sentences with a mean of 9.3 words, averaging 53.1 characters per review. Word count varies slightly by class, negative reviews had a mean of 10.8 words, neutral 8.2 words, and positive 8.8 words. Pairwise Jaccard similarity between class vocabulary after stop word removal was 0.27 till 0.33 across all three class pairs. Negative reviews had the largest vocabulary, sitting at 3613 unique words. Positive second with 2498 unique words, and neutral last at 2491 unique words. The top 10 TF-IDF weighted bigrams or trigrams were examined for negative reviews, positive reviews, and neutral reviews. They can be found in Fig. 2, Fig. 3, and Fig. 4, respectively.

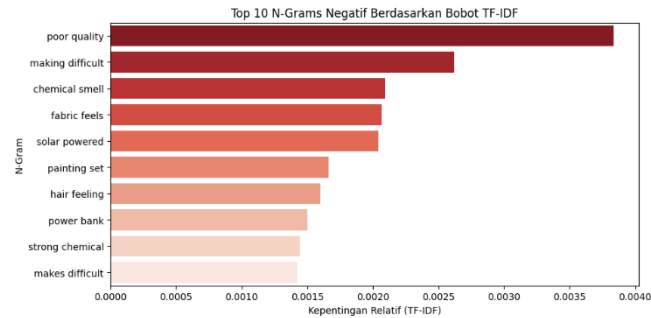


Fig. 2. Top 10 TF-IDF weighted n-grams for negative reviews

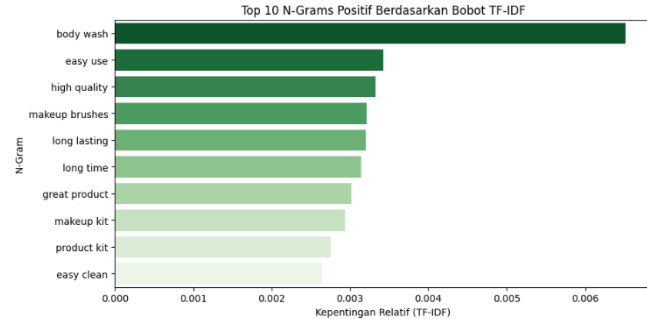


Fig. 3. Top 10 TF-IDF weighted n-grams for positive reviews

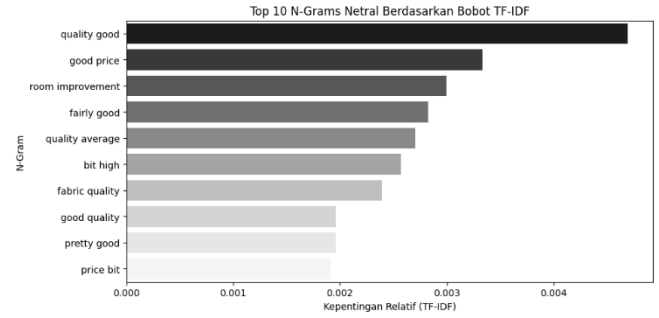


Fig. 4. Top 10 TF-IDF weighted n-grams for neutral reviews

Following the EDA findings, preprocessing was performed to clean the text in the dataset. A basic cleaning function was applied to the text: conversion to lowercase, removal of non-ASCII characters, removal of URLs, removal of digits, removal of punctuation and non-word characters, and removal of excessive whitespace [18]. Two preprocessing tracks were made, one for the classical machine learning models (Naive Bayes, Logistic Regression, XGBoost) and one for BERT. The classical machine learning models (Naive Bayes, Logistic Regression, and XGBoost) worked on text that was further processed using tokenization, English stop word removal, Porter stemming, and removal of single character tokens. BERT used text that only went through the basic cleaning mentioned at the beginning since BERT's sub word tokenizer and contextual representation rely on natural, unmodified language making any further preprocessing such as removal of stop words or stemming negatively impact the performance of the model [19]. The cleaning process is demonstrated in Table I below, text is the unprocessed review, text_clean_base is text that went through basic cleaning, and text_classical is text_clean_base that went through classical cleaning.

TABLE I. COMPARISON OF TEXT PREPROCESSING

text	text_clean_base	text_classical
Great but disappointingly small	great but disappointingly small	great disappointingly small
The design is very fashionable	the design is very fashionable	design fashion
Suitable for the whole family	suitable for the whole family	suitabl whole famili
Playdo is dry and cannot be used	playdo is dry and cannot be used	playdo dri use
No smell which is good	no smell which is good	smell good

The sentiment labels were then encoded to integers via LabelEncoder. The cleaned dataset amounting 8587 rows (34.4% positive, 33.4% negative, 32.1% neutral) was then split into 80% training, 10% validation, and 10% test sets using stratified sampling. For the classical models, TF-IDF from scikit-learn was used because it yields improved classification performance over alternative feature weighting schemes like basic n-gram counts [20]. TF-IDF is initialized for unigrams and bigrams, with a maximum vocabulary of 10000 features, sublinear term-frequency scaling, and document frequency filtering (min_df=2, max_df=0.95) to exclude rare and common terms. It was fit on training data and then applied to validation and test sets to prevent data leakage. For BERT, BertTokenizerFast from bert-base-

uncased was used to tokenize text into subword token IDs with padding up to a maximum length of 48 tokens as 99% percent of words fall below 40 words and an attention mask. Data was wrapped in a PyTorch Dataset class with a batch size of 16 for training and 32 for validation and testing.

D. Model Training

Once our data had been preprocessed and feature extraction completed, we proceeded to train four text classification models based on the prior exploratory data analysis, preprocessing, and feature extraction steps. These consist of three classical machine learning models and one model based on a transformer (deep learning) architecture. The details of each model are as follows.

The first model we used was Logistic Regression, employed as a baseline linear classifier model [21]. The model learns a weight vector for each TF-IDF feature by optimizing the negative log-likelihood function through gradient descent. For our multi-label case (Positive, Neutral, Negative), we used the multinomial extension, which replaces the sigmoid function with a softmax function so that the output can represent the probability distribution across all three classes simultaneously. This model was chosen because it can serve as a competitive baseline for TF-IDF-based text classification tasks [21], and it offers transparency through feature coefficients that can be directly interpreted. The logistic regression model was initialized with the liblinear solver and a maximum of 1000 iterations. We performed hyperparameter tuning using GridSearchCV with 5-fold cross-validation and `f1_macro` scoring across combinations of the regularization parameter `C` [0.001, 0.01, 0.1, 1, 10, 100] and penalty ['l1', 'l2'], identifying the optimal parameters as `C = 10` and penalty = `l2`.

The second model we used was Multinomial Naive Bayes, selected because this model is explicitly designed for frequency-based text data [22]. Naive Bayes is a probabilistic model that applies Bayes' theorem to estimate the probability of each class. Its architecture models two components: the prior class probability $P(c)$, computed from class frequency in the training data, and the word likelihood probability $P(w|c)$, computed from word occurrence frequency per class. At prediction time, both components are multiplied for every word in the text using the formula:

$$P(c|text) = \frac{\prod P(w_i|c) \times P(c)}{P(text)} \quad (1)$$

After this calculation, the class with the highest probability is selected. The model is termed "naive" because it assumes conditional independence between words. We chose the Multinomial variant because it models word occurrence frequency rather than mere presence or absence, making it more compatible with TF-IDF representation [22]. We applied GridSearchCV (5-fold CV, `f1_macro` scoring) over alpha values [0.001, 0.01, 0.1, 0.5, 1.0, 2.0], yielding an optimal alpha = 0.5 that balances smoothing strength with discriminative power.

The third model we used was XGBoost, chosen to represent a tree-based ensemble approach [23, 24]. Architecturally, XGBoost builds a sequence of decision trees, where each new tree focuses on correcting the residual error of the previous tree through gradient descent in function space (gradient boosting). XGBoost's objective function combines

two components: a loss function and explicit L1/L2 regularization on the weights and number of leaf nodes [24]. Unlike conventional gradient boosting, XGBoost is more resistant to overfitting due to this built-in regularization [24]. We used `objective = multi:softmax` for three-class classification. RandomizedSearchCV (`n_iter=30`, 5-fold CV, `f1_macro` scoring) was used to tune critical hyperparameters such as `n_estimators`, `max_depth`, `learning_rate`, `subsample`, `colsample_bytree`, `min_child_weight`, and `gamma`. We determined the best configuration by applying `tree_method = hist` for efficient sparse matrix processing.

The final model we used was BERT (bert-base-uncased), chosen to represent the transformer-based deep learning approach due to its ability to generate contextual representations through a bidirectional self-attention mechanism [25]. BERT's architecture consists of 12 transformer encoder layers, each with 12 attention heads and a hidden size of 768 dimensions, totaling approximately 110 million trainable parameters. The self-attention mechanism allows each token to attend to every other token in the sequence bidirectionally, capturing long-range semantic dependencies that cannot be modeled by TF-IDF [25]. We used BertForSequenceClassification with `num_labels = 3`, fine-tuned for 4 epochs using AdamW (LR = $2e-5$, `weight_decay = 0.01`) and a linear warmup-decay scheduler (`warmup_proportion = 10%` of total training steps), and applied gradient clipping (`max_norm = 1.0`). The checkpoint with the highest validation accuracy was saved as the final model.

TABLE II. MODEL HYPERPARAMETER CONFIGURATIONS

Model	Hyperparameters	Search Strategy
Logistic Regression	<code>C = 10</code> , <code>penalty = l2</code>	GridSearchCV (5-fold)
Naïve Bayes	<code>alpha = 0.5</code>	GridSearchCV (5-fold)
XGBoost	RandomizedSearch	RandomizedSearchCV (30 iter, 5-fold)
BERT	<code>lr=2e-5</code> , <code>batch 16</code> , <code>epochs = 4</code>	Manual

E. Model Evaluation

Evaluation of the four models' performance was conducted on the test set, which had previously been split from the training and validation data. Each metric used was selected by considering the characteristics of the dataset, which consists of three relatively balanced classes, as well as the need to assess model performance fairly across each sentiment class. The metrics used are described as follows.

First, we used the confusion matrix, a 3×3 matrix that summarizes the distribution of the model's predictions against the actual labels for the three classes (positive, neutral, negative). In the confusion matrix, the rows represent the actual class, while the columns represent the model's predicted class. Diagonal elements reflect correct predictions, while off-diagonal elements reflect prediction errors. This matrix comprises four base values, True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN), which together form the foundation of all other evaluation metrics [26]. We selected the confusion matrix because it provides insight not only into how often the model is correct, but also into the specific error patterns between classes. Confusion

matrix analysis is essential for understanding whether the model tends to confuse the neutral class with the positive or negative class, which represents a key challenge in three class sentiment classification [26].

The second metric we used was accuracy, which measures the overall proportion of correct predictions out of all test samples, defined as:

$$Accuracy = \frac{(TP+TN)}{(TP+TN+FP+FN)} \quad (2)$$

We chose accuracy as a baseline metric that is easy to interpret and is frequently used in model comparisons. The class distribution in our dataset is nearly balanced (34.7% Positive, 33.2% Negative, 32.1% Neutral), which makes accuracy remain representative and appropriate. On the other hand, when a dataset is imbalanced, high accuracy can be achieved simply by consistently predicting the majority class [27].

The third metric we used was Precision, which measures how precise the **model** is when predicting a given class, in other words, out of all predictions made for class x, what percentage are actually class x. Precision is defined as:

$$Precision = \frac{TP}{(TP+FP)} \quad (3)$$

We used precision to assess how trustworthy the model's positive predictions are. In the context of online shop review sentiment analysis, a high False Positive rate means the system too frequently misclassifies negative reviews as positive, which could mislead business decision-making. Precision is reported per class to identify which class most often yields imprecise predictions [27].

The fourth metric we used was Recall, which measures how completely the model captures samples of a given class in other words, out of all samples that actually belong to class x, what percentage are successfully detected. Recall is defined as:

$$Precision = \frac{TP}{(TP+FN)} \quad (4)$$

Recall complements precision by measuring the model's ability to avoid missing samples from a given class. A low recall on the negative class, for instance, means that many negative reviews go undetected. Recall is reported per class to assess the completeness of the model's detection for each sentiment.

The fifth metric we used was the F1-Score, the harmonic mean of precision and recall that balances the two, defined as:

$$F1 = 2 \times \frac{(Precision \times Recall)}{(Precision + Recall)} \quad (5)$$

We chose the F1-Score to combine precision and recall into a single value, making it more informative than either metric alone. We use this averaged value because it is more sensitive to imbalances between precision and recall. If a model has very high precision but very low recall, or vice versa, it will receive a low F1 score, which more accurately reflects its true quality [27].

The sixth metric we used was the Macro F1-Score, the unweighted average of the F1-Scores across all classes, defined as:

$$Macro\ F1 = \frac{(F1_Negative + F1_Neutral + F1_Positive)}{3} \quad (6)$$

We chose the Macro F1-Score as our primary metric for comparing models in our evaluation because it treats every class equally, regardless of differences in sample size, ensuring that performance on the Neutral class, which tends to be the most difficult to classify is not masked by strong performance on the other classes. Furthermore, using Macro F1 as the primary metric in comparative multi-model studies has become common practice in NLP, as it provides a fairer and more consistent picture of performance [28].

The seventh and final metric we used was the **Weighted F1-Score**, which computes the F1-Score average weighted by the number of samples in each class, defined as:

$$Macro\ F1 = \frac{\sum (F1_class \times Support_class)}{Sample\ totals} \quad (7)$$

We used the Weighted F1-Score to complement the Macro F1-Score by providing an additional perspective that accounts for the actual proportion of each class within the dataset. Together, Macro F1 and Weighted F1 provide a more comprehensive evaluation picture: Macro F1 assesses the balance of performance across classes, while Weighted F1 reflects the model's overall performance relative to the true data distribution [28].

IV. RESULTS AND ANALYSIS

A. System Implementation

The whole experimental pipeline was implemented in Python using Jupyter Notebook (FullCodeNLP.ipynb). Saved artifacts include: tfidf_vectorizer.pkl, label_encoder.pkl, lr_model.pkl, nb_model.pkl (all serialized using pickle), xgb_model.json (native XGBoost JSON format), and bert_best/ (HuggingFace checkpoint folder containing config.json, model.safetensors, tokenizer.json, tokenizer_config.json).

The CheckShop web app was built using Streamlit and implemented in App.py. The application loads all four models at startup to prevent redundant loading using @st.cache_resource. Users enter their review text in English and the system after processing the text will output predictions and class probabilities from all four models simultaneously. The results will include for each model, a probability bar chart for each class and a combined detailed probability table. There will also be a banner that indicates whether all models agreed on the same class or what the majority agreed on.

B. Experimental Results

The hyperparameter tuning found best for Logistic Regression was {'C': 10, 'penalty': 'l2'}. The results of the model on the validation set can be found in Table III below.

TABLE III. CLASSIFICATION REPORT FOR LOGISTIC REGRESSION (VAL)

Logistic Regression (Val)	Precision	Recall	F1-score
---------------------------	-----------	--------	----------

Negative	0.89	0.90	0.89
Neutral	0.81	0.76	0.78
Positive	0.82	0.87	0.85
Accuracy			0.84
Macro Average	0.84	0.84	0.84
Weighted Average	0.84	0.84	0.84

The confusion matrix for Logistic Regression on the validation set can be seen below in Fig. 5.

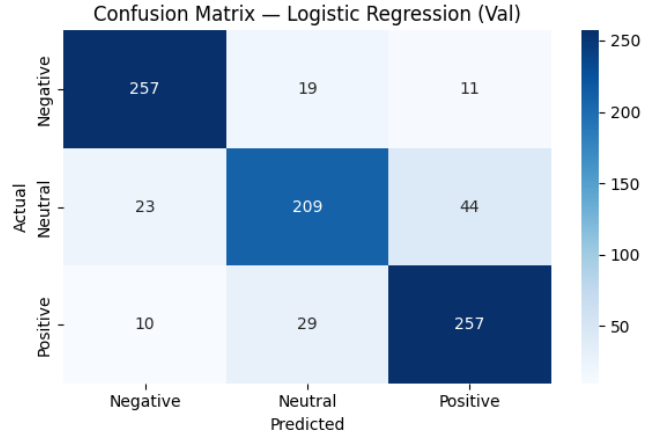


Fig. 5. Confusion Matrix for Logistic Regression on Validation set

The results of the performance of Logistic Regression on the test set of 859 samples can be found in Table IV below.

TABLE IV. CLASSIFICATION REPORT FOR LOGISTIC REGRESSION (TEST)

Logistic Regression (Test)	Precision	Recall	F1-score
Negative	0.86	0.87	0.86
Neutral	0.79	0.72	0.75
Positive	0.81	0.87	0.84
Accuracy			0.82
Macro Average	0.82	0.82	0.82
Weighted Average	0.82	0.82	0.82

The confusion matrix for Logistic Regression on the test set can be seen below in Fig. 6.

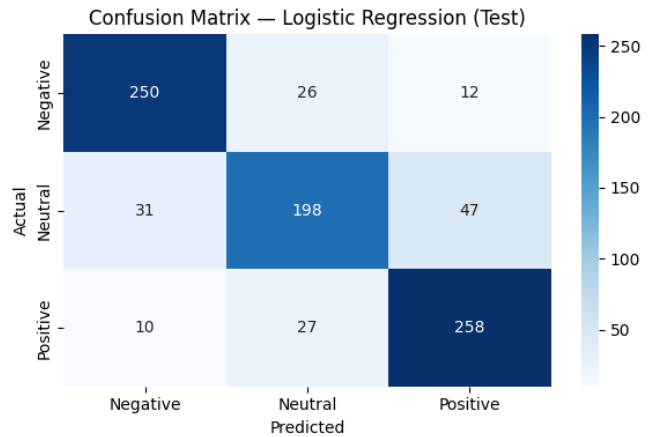


Fig. 6. Confusion Matrix for Logistic Regression on Test set

The hyperparameter tuning found best for Naive Bayes was {'alpha': 0.5}. The results of the model on the validation set can be found in Table V below.

TABLE V. CLASSIFICATION REPORT FOR NAIVE BAYES (VAL)

Naive Bayes (Val)	Precision	Recall	F1-score
Negative	0.87	0.90	0.88
Neutral	0.80	0.74	0.77
Positive	0.83	0.87	0.85
Accuracy			0.84
Macro Average	0.84	0.84	0.83
Weighted Average	0.84	0.84	0.84

The confusion matrix for Naive Bayes on the validation set can be seen below in Fig. 7.

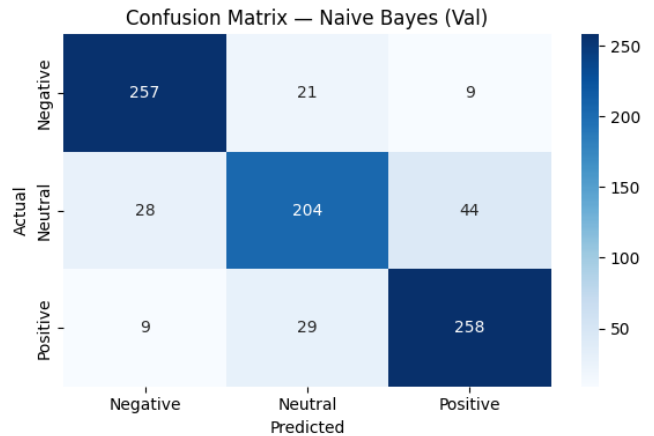


Fig. 7. Confusion Matrix for Naive Bayes on Validation set

The results of the performance of Naive Bayes on the test set of 859 samples can be found in Table VI below.

TABLE VI. CLASSIFICATION REPORT FOR NAIVE BAYES

Naive Bayes (Test)	Precision	Recall	F1-score
--------------------	-----------	--------	----------

Negative	0.85	0.87	0.86
Neutral	0.77	0.72	0.74
Positive	0.83	0.86	0.85
Accuracy			0.82
Macro Average	0.82	0.82	0.82
Weighted Average	0.82	0.82	0.82

The confusion matrix for Naive Bayes on the test set can be seen below in Fig. 8.

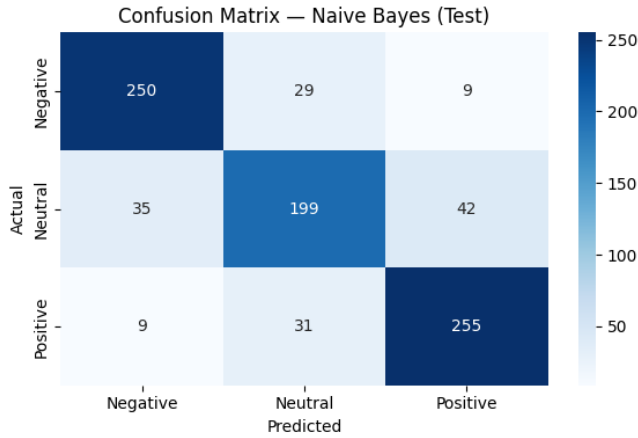


Fig. 8. Confusion Matrix for Naive Bayes on Test set

The hyperparameter tuning found best for XGBoost was {'subsample': 1.0, 'n_estimators': 500, 'min_child_weight': 1, 'max_depth': 6, 'learning_rate': 0.1, 'gamma': 0.5, 'colsample_bytree': 0.6}. The best iteration was found to be the 499th. The results of the performance of XGBoost on the test set of 859 samples can be found in Table VII below.

TABLE VII. CLASSIFICATION REPORT FOR XGBOOST

XGBoost (Test)	Precision	Recall	F1-score
Negative	0.90	0.80	0.85
Neutral	0.72	0.74	0.73
Positive	0.76	0.83	0.80
Accuracy			0.79
Macro Average	0.79	0.79	0.79
Weighted Average	0.79	0.79	0.79

The confusion matrix for XGBoost on the test set can be seen below in Fig. 9.

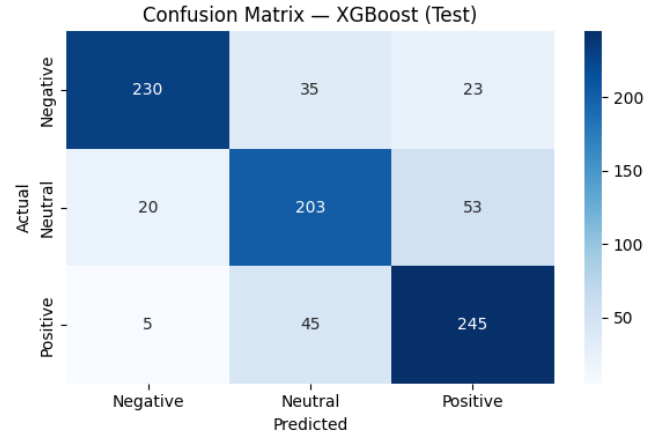


Fig. 9. Confusion Matrix for XGBoost on Test set

The top 20 most important features for XGBoost can be seen in Fig. 10 below.

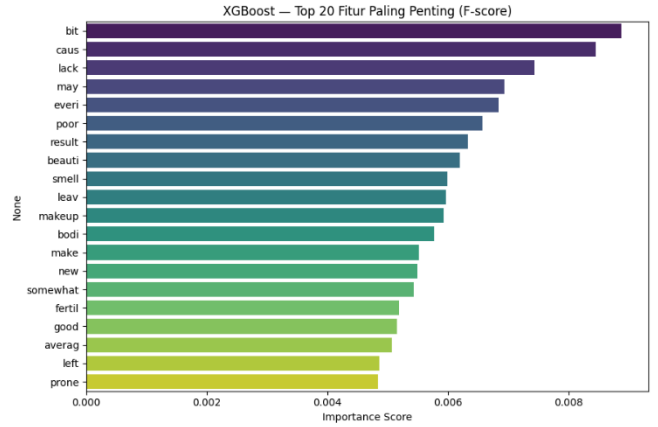


Fig. 10. Top 20 Most Important Features for XGBoost

The BERT model's loss per epoch can be seen in Fig. 11.

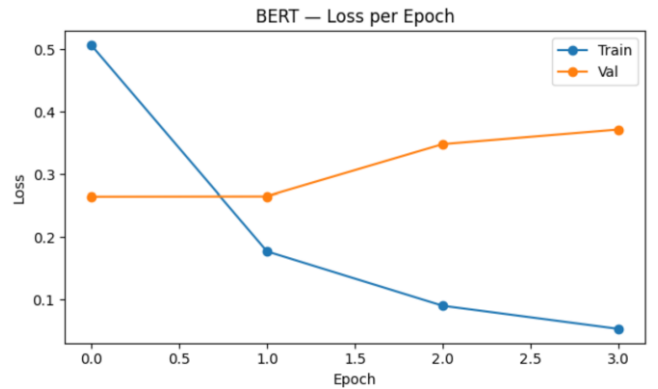


Fig. 11. BERT Loss per Epoch

The BERT model's accuracy per epoch can be seen in Fig. 12.

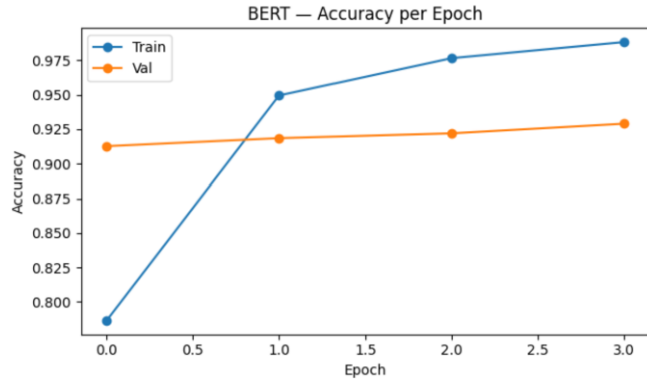


Fig. 12. BERT accuracy per Epoch

The results of the performance of BERT on the test set of 859 samples can be found in Table VIII below.

TABLE VIII. CLASSIFICATION REPORT FOR BERT

BERT (Test)	Precision	Recall	F1-score
Negative	0.97	0.94	0.95
Neutral	0.94	0.91	0.93
Positive	0.92	0.98	0.95
Accuracy			0.94
Macro Average	0.94	0.94	0.94
Weighted Average	0.94	0.94	0.94

The confusion matrix for BERT can be seen below in Fig. 13.

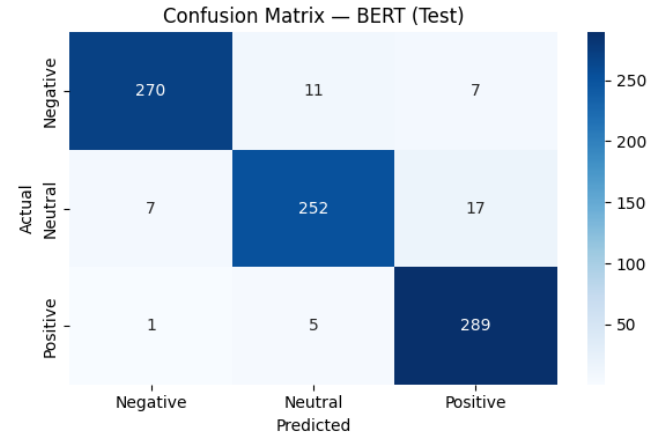


Fig. 13. Confusion Matrix for BERT on Test set

23 samples were found to be wrongly predicted by all four models, Fig. 14 below shows some of them.

	text_original	true	pred_lr	pred_nb	pred_xgb	pred_bert
1	dji rc remote controller for dji air and dji m...	Negative	Positive	Positive	Positive	Positive
2	it becomes more of a decorative piece than a u...	Negative	Neutral	Neutral	Neutral	Neutral
3	will smooth easily	Neutral	Positive	Positive	Positive	Positive
4	the quality of the beads is average	Negative	Neutral	Neutral	Neutral	Neutral
5	of course there are simplistic negative points	Negative	Neutral	Neutral	Neutral	Neutral
6	buying gold is fun but expensive	Negative	Positive	Positive	Positive	Neutral
7	the travel essential has become anything but	Negative	Positive	Positive	Positive	Neutral

Fig. 14. Wrongly predicted reviews by all four models

C. Discussion

The results confirm that BERT achieved the best performance among all four models. This is seen by BERT's macro F1 being roughly 12 percent above the best classical model baseline (94.4% vs 82% Logistic Regression). This gap is attributable to BERT's algorithm that captures long-range dependencies and semantic nuances that TF-IDF's bag-of-words assumption can't represent. Logistic Regression's competitive performance though can be explained by the addition of bigrams to the TF-IDF which somewhat makes up for its inability to capture context and semantic nuances.

The lackluster performance of XGBoost is unusual given its strength on structured data. This could be because of the sparsity of TF-IDF feature matrices since gradient-boosted trees aren't designed for high-dimensional sparse inputs and may instead overfit feature interactions that aren't meaningful. Dense embedding features like averaged word2vec or fastText vectors can be explored as alternatives to TF-IDF for XGBoost.

BERT's remarkable performance comes at the cost of computational power needed. Fine-tuning the model required GPU access and inference on the Streamlit application is slower for BERT compared to the classical models. The model file size being quite large (418 MB) also made deployment a challenge, requiring Git LFS. Classical model files on the other hand even when combined were under 1 MB, making them super lightweight and computationally cheaper to run than BERT. For deployment scenarios that require real-time inference and minimal compute power, Logistic Regression is a strong alternative to BERT. Its interpretability through looking at feature coefficients is another plus compared to BERT.

Analysis shows that reviews containing contrastive conjunctions such as "but", "however", and "although" are strong predictors of the Neutral class as 96.5% of sentences combining negation with a contrastive conjunction and 93.5% of contrastive sentences are generally labeled Neutral. Reviews with only negation however skew toward Negative.

V. CONCLUSION AND FUTURE WORK

This research produced CheckShop, a comparative multi-model sentiment analysis system that takes product or service reviews in English and outputs predicted sentiment from Naive Bayes, Logistic Regression, XGBoost, and BERT. The dataset that was used to train all four models comes from [17]. Experiments showed BERT achieving the highest performance with a macro F1 of 94.4%, outperforming the best classical model, Logistic Regression, by roughly 12%. Among the classical models, Logistic Regression and Naive Bayes performed similarly yet better than XGBoost on high-dimensional TF-IDF feature space.

For future work, several directions are proposed. First, the dataset scope can be expanded to include reviews from other cultural or linguistic backgrounds as the current dataset is limited to Bengali e-commerce. Second, the sentiment classification can be developed to differentiate between different product aspects like delivery and quality. Third, the trained models will not update as new data comes in, leading to model drift in due time. Continuous retraining mechanisms can be implemented to address model drift. Lastly,

integrating explainable AI techniques like SHAP or LIME can be implemented to enhance model transparency for users.

REFERENCES

- [1] M. E. Alzahrani, T. H. H. Aldhyani, S. N. Alsabari, M. M. Althobaiti, and A. Fahad, "Developing an Intelligent System with Deep Learning Algorithms for Sentiment Analysis of E-Commerce Product Reviews," *Computational Intelligence and Neuroscience*, vol. 2022, art. no. 3840071, May 2022, DOI: 10.1155/2022/3840071
- [2] K. Koruyan, "Sentiment Analysis of Online Customer Reviews Using Modern Natural Language Processing Approaches," *Research Gate*, 2025. DOI: 10.29157/etusbod.1711500
- [3] Firdaus et al., "Application of Sentiment Analysis as an Innovative Approach to Policy Making: A Review," *Journal of Robotics and Control (JRC)*. DOI: 10.18196/jrc.v5i6.22573
- [4] M. Wongkar and A. Angdresey, "Sentiment Analysis Using Naive Bayes Algorithm Of The Data Crawler: Twitter," *Research Gate*, 2019. DOI: 10.1109/ICIC47613.2019.8985884
- [5] D. Manuel-Ilie, A. Pitic, and R. G. Cretulescu, "Sentiment Analysis Using Bert Model," *Research Gate*, 2023. DOI: 10.2478/ijasitels-2023-0007
- [6] M. S. Islam, L. Xiangdong, and J. Ahmed, "BERT: advancements in language understanding for different NLP tasks: challenges and future perspectives," *Journal of Electrical Systems and Inf Techno*, 2026. DOI: 10.1186/s43067-026-00341-1
- [7] E. Cambria, D. Das, S. Bandyopadhyay, and A. Feraco, "Affective Computing and Sentiment Analysis," Springer Nature, 2017. DOI: 10.1007/978-3-319-55394-8_1
- [8] Abighail et al., "Sentiment Analysis E-commerce Review," *Science Direct*, 2023. DOI: 10.1016/j.procs.2023.10.613
- [9] J. I. W. Siregar, A. P. U. Siahaan, M. Iqbal, D. Nasution, and R. F. Wijaya, "Sentiment Analysis Classification of E-commerce User Reviews Using Natural Language Processing (NLP) and Support Vector Machine (SVM) Methods," *Bulletin of Information Technology*, 2025. DOI: 10.47065/bit.v5i2.2018
- [10] Q. Wang, "Support Vector Machine Algorithm in Machine Learning," *Research Gate*, 2022. DOI: 10.1109/ICAICA54878.2022.9844516
- [11] P. S. Reddy, D. R. Sri, S. R. Cheerka, and S. Shaik, "Sentimental Analysis using Logistic Regression," *Research Gate*, 2021. DOI: 10.9790/9622-1107023640
- [12] M. Wanyonyi, Z. N. Morris, F. M. Musyoka, and D. M. Kitavi, "Balancing Interpretability and Accuracy: A Comparative Evaluation of Logistic Regression and Machine Learning Approaches for Coronary Heart Disease Risk Prediction in Low-Resource Settings," *Research Gate*, 2021. DOI: 10.36227/techrxiv.175393687.78366329/v1
- [13] T. Chen and C. Guestin, "XGBoost: A Scalable Tree Boosting System," *ACM*, 2016. DOI: 10.1145/2939672.2939785
- [14] X. Lin, "Sentiment Analysis of E-commerce Customer Reviews Based on Natural Language Processing," in *Proc. 2020 2nd Int. Conf. Big Data Artif. Intell. (ISBDAI'20)*, ACM, pp. 32-36, 2020. DOI: 10.1145/3436286.3436293
- [15] R. Kosasih and A. Alberto, "Sentiment analysis of game product on shopee using the TF-IDF method and naive bayes classifier," *ILKOM Jurnal Ilmiah*, vol. 13, no. 2, pp.101-109, Aug. 2021. DOI: 10.33096/ilkom.v13i2.721.101-109
- [16] S. A. H. Bahtier, C.K.Dewa, and A.Luthfi, "Comparison of Naïve Bayes and Logistic Regression in Sentiment Analysis on Marketplace Reviews Using Rating-Based Labeling," *Journal of Information Systems and Informatics*, vol. 5, no. 3, pp.915-927, Aug. 2023. DOI: 10.51519/journalisi.v5i3.539
- [17] U. A. ayman, T. Akter, M. T. A. Akash, and S. Z. Mridul, "BanglaEcomReviewCorpus: A Resource for E-Commerce Product Review Sentiment Analysis," *Mendeley Data*, v2, Jul. 2025, DOI: 10.17632/kkzfrvhbhp.2
- [18] S. J. Baroi, N. Singh, R. Das, and T. D. Singh, "NITS-Hinglish-SentiMix at SemEval-2020 Task 9: Sentiment Analysis for Code-Mixed Social Media Text Using an Ensemble Model," in *Proc. 14th Workshop Semantic Evaluation*, Dec. 2020, pp. 1298–1303, DOI: 10.18653/v1/2020.semeval-1.175.
- [19] M. M. Abdelgwad, T. H. A. Soliman, and A. I. Taloba, "Arabic aspect sentiment polarity classification using BERT," *Journal of Big Data*, Dec. 2022, DOI: 10.1186/s40537-022-00656-6.
- [20] M. Das, K. Selvakumar, and P. J. A. Alphonse, "A Comparative Study on TF-IDF feature Weighting Method and its Analysis using Unstructured Dataset," *Proc. 5th Int. Conf. Comput. Linguistics Intell. Syst. (COLINS)*, Apr. 2021. DOI: 10.48550/arXiv.2308.04037
- [21] Shetty, A.M., Aljunid, M.F., Manjaiah, D.H., "Sentiment Exploring on Feedback of E-commerce Data Using Machine Learning Algorithms," Springer, 2024. DOI: 10.1007/978-981-99-7622-5_8
- [22] Dewi, C., Chen, R.-C., Christanto, H.J., & Cauteruccio, F., "Multinomial Naïve Bayes Classifier for Sentiment Analysis of Internet Movie Database," *Vietnam Journal of Computer Science*, 2023. DOI: 10.1142/S2196888823500100
- [23] Talaat, A.S., "Sentiment analysis classification system using hybrid BERT models," *Journal of Big Data*, 2023. DOI: 10.1186/s40537-023-00781-w
- [24] Lin, J., Yaseen, Z.M., & Omondi, E.O., "Application of machine learning in predicting consumer behavior and precision marketing," *PLOS ONE*, 2025. DOI: 10.1371/journal.pone.0321854
- [25] Fang, H., Jiang, G., & Li, D., "Sentiment analysis based on Chinese BERT and fused deep neural networks for sentence-level Chinese e-commerce product reviews," *Systems Science & Control Engineering*, 2022. DOI: 10.1080/21642583.2022.2123060
- [26] S. Swaminathan et al., "Confusion matrix-based performance evaluation metrics," *African Journal of Biomedical Research*, vol. 27, no. 4S, pp. 4023–4031, Nov. 2024. DOI: 10.53555/AJBR.v27i4S.4345
- [27] J. H. Cabot and E. G. Ross, "Evaluating prediction model performance," *Surgery*, vol. 174, no. 3, pp. 723–726, Sep. 2023, DOI: 10.1016/j.surg.2023.05.023
- [28] T. Zhang, I. C. Irsan, F. Thung, and D. Lo., "Revisiting Sentiment Analysis for Software Engineering in the Era of Large Language Models," *ACM Trans*, 2025. DOI: 10.1145/3697009

APPENDIX

A. Team Contribution Statements

Jason Montolalu: Conceptualization, Software, Data Curation, Formal Analysis, Investigation, Validation, Writing - Original Draft, Writing - Review & Editing, Methodology, Visualization. **Rakha Adinata Zavier:** Conceptualization, Software, Writing - Original Draft, Writing - Review & Editing, Methodology, Visualization, Formal Analysis, Investigation. **Muhammad Ridho Prakoso:** Conceptualization, Formal Analysis, Investigation, Writing - Original Draft, Writing - Review & Editing, Methodology, Visualization. **Catherine Zaneta Adji:** Conceptualization, Formal Analysis, Investigation, Writing - Original Draft, Writing - Review & Editing, Methodology, Visualization. **Mohammad Faisal Riftiarsyid:** Supervision.

We use Git to perform shared tasks, where if one of us finds a bug or an issue in our application, we update it via Git (git commit). We also use Google Colab to collaborate on code creation and to perform quality checks regarding the code made by one another.

We declare that the distribution of team member contributions listed above has been structured fairly, transparently, and in accordance with the respective duties and responsibilities of each member during the project execution process.

All information regarding roles and contributions has been written truthfully based on the work actually performed, without exaggerating or diminishing the contribution of any party. Each member is responsible for the tasks they have completed and agrees to the distribution of contributions.

B. Screenshoot

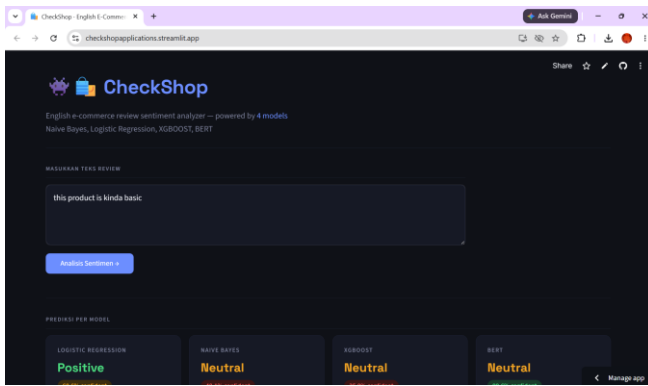


Fig. 15. Application UI Screenshoot (1)

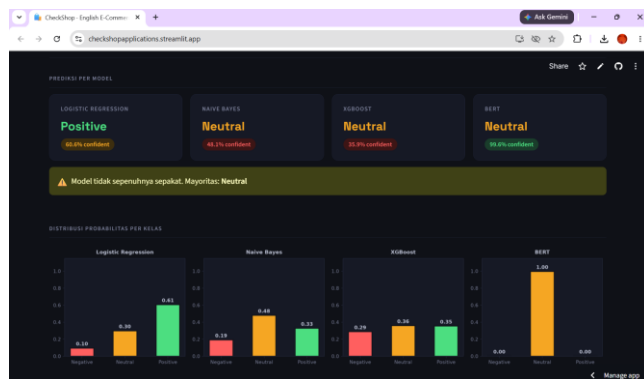


Fig. 16. Application UI Screenshoot (2)

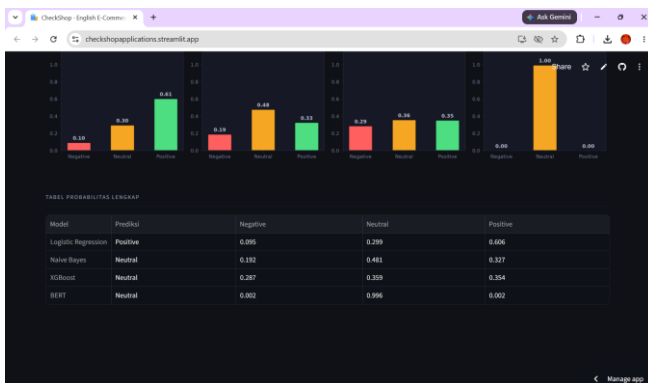


Fig. 17. Application UI Screenshoot (3)

C. Snippet Code

Libraries

```
# Import libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import re, pickle, warnings
import numpy as np
import nltk

from sklearn.feature_extraction.text
import TfidfVectorizer
```

```
from sklearn.linear_model import
LogisticRegression
from sklearn.metrics import(
    classification_report,
    confusion_matrix,
    accuracy_score, f1_score
)
from sklearn.naive_bayes import
MultinomialNB
from sklearn.model_selection import
train_test_split
from sklearn.preprocessing import
LabelEncoder
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.tokenize import
word_tokenize
import xgboost as xgb

nltk.download('stopwords', quiet=True)
nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)
```

Load Dataset

```
# Display dataset original
df =
pd.read_csv('BanglaEcomReviewCorpus.cs
v', sep=';')

print(f"Bentuk data: {df.shape}")
print(f"\nDistribusi sentiment:")
print(df['Sentiment'].value_counts())
df.head()
```

Exploratory Data Analysis

```
# Display distribusi kelas
plt.figure(figsize=(8, 5))
counts =
df['Sentiment'].value_counts()
total = len(df)

ax = sns.countplot(
    data=df,
    x='Sentiment',
    hue='Sentiment',
    palette='viridis',
    order=counts.index,
    legend=False
)
```

Data Cleaning

```

stemmer = PorterStemmer()
stop_words =
set(stopwords.words('english'))

# Bikin function untuk clean text
untuk model Naive Bayes, Logistic
Regression, dan XGBoost
def clean_text_classical(text):
    text = clean_text_base(text)
    tokens = word_tokenize(text)
    # Hapus stopwords
    tokens = [t for t in tokens if t
not in stop_words]
    # Stemming
    tokens = [stemmer.stem(t) for t in
tokens]
    # Hapus token terlalu pendek (1
karakter)
    tokens = [t for t in tokens if
len(t) > 1]
    return ' '.join(tokens)

df['text_classical'] =
df['text'].apply(clean_text_classical)

print('Contoh perbandingan:')
df[['text', 'text_clean_base',
'text_classical']].head(8)

```

- Preprocessing

```

# Label encoding untuk mengubah
sentimen menjadi integer
le = LabelEncoder()
df['label_encoded'] =
le.fit_transform(df['Sentiment'])

print('Mapping label:')
for i, cls in enumerate(le.classes_):
print(f' {cls} -> {i}')

```

- Train Model

```

from sklearn.model_selection import
GridSearchCV

# Inisialisasi model Logistic
Regression
base_lr =
LogisticRegression(max_iter=1000,
solver='liblinear', random_state=42)

```

```

# Inisialisasi parameter grid search
param_grid = {
    'C': [0.001, 0.01, 0.1, 1, 10,
100],
    'penalty': ['l1', 'l2']
}

# Hyperparameter tuning menggunakan
GridSearch
print('— Hyperparameter Tuning —')
grid_search = GridSearchCV(base_lr,
param_grid, cv=5, scoring='f1_macro',
n_jobs=-1)
grid_search.fit(X_train_tfidf,
y_train)

```

- Evaluation

```

def evaluate_model(name, y_true,
y_pred, class_names=CLASS_NAMES):
    acc = accuracy_score(y_true,
y_pred)
    macro_f1 = f1_score(y_true,
y_pred, average='macro')

    print(f'{'='*55}')
    print(f' {name}')
    print(f'{'='*55}')
    print(f' Accuracy : {acc:.4f}')
    print(f' Macro F1 :
{macro_f1:.4f}')
    print()
    print(classification_report(y_true
, y_pred, target_names=class_names))

```